DPJ

<Test Plan>

Version <1.0>

Revisions

Date	Version	Description	Author
06/12/2025	1.0	Complete sections from 6 to 10	José Ángel Sotoca Pulpón

Index

Test plan

To prescribe the scope, approach, resources, and schedule of the testing activities. To identify the items being tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task, and the risks associated with this plan.

1. Test plan identifier [Daniel]

Specify the unique identifier assigned to this test plan.

2. Introduction [Daniel]

Summarize the software items and software features to be tested. The need for each item and its history may be included.

References to the following documents, when they exist, are required in the highest level test plan:

a) Project authorization;
b) Project plan;
c) Quality assurance plan;
d) Configuration management plan;
e) Relevant policies;
f) Relevant standards.

In multilevel test plans, each lower-level plan must reference the next higher-level plan.

3. Test items [Daniel]

Identify the test items including their version/revision level. Also specify characteristics of their transmittal media that impact hardware requirements or indicate the need for logical or physical transformations before testing can begin (e.g., programs must be transferred from tape to disk).

Supply references to the following test item documentation, if it exists:

g) Requirements specification;
h) Design specification;
i) Users guide;
j) Operations guide;
k) Installation guide.

Reference any incident reports relating to the test items.

Items that are to be specifically excluded from testing may be identified.

4. Features to be tested [Daniel]

Identify all software features and combinations of software features to be tested. Identify the test design specification associated with each feature and each combination of features.

//Use Cases usadas

5. Features not to be tested [Daniel]

Identify all features and significant combinations of features that will not be tested and the reasons.

//Use Cases no usadas

6. Approach [José Ángel]

Describe the overall approach to testing. For each major group of features or feature combinations, specify the approach that will ensure that these feature groups are adequately tested. Specify the major activities, techniques, and tools that are used to test the designated groups of features.

The approach should be described in sufficient detail to permit identification of the major testing tasks and estimation of the time required to do each one.

Specify the minimum degree of comprehensiveness desired. Identify the techniques that will be used to judge the comprehensiveness of the testing effort (e.g., determining which statements have been executed at least once). Specify any additional completion criteria (e.g., error frequency). The techniques to be used to trace requirements should be specified.

Identify significant constraints on testing such as test item availability, testing resource availability, and deadlines.

Overall Approach to Testing

The system's test cases are defined based on its functional specifications (use cases), every test case is based on and corresponds to an application (system) use case. The two primary workflows (create a plan and purchase) are tested complete to verify that all interactions on the front-end with a user also trigger the appropriate logic on the back-end and that the data submitted via the plan form is both validated and saved correctly as well as errors are handled appropriately. Additionally, the tests included are positive flows, negative flows and edge cases.

Major Feature Groups and Testing Approach

1. Plan Creation Workflow:

The features tested include creating a plan using a form, selecting classes, confirming the plan and validating errors. Functional tests will simulate all expected user interaction with the front-end via Selenium WebDriver on a computer. Validation of the back-end processing will be completed by utilizing an Application Programming Interface (API) and performing a database transaction (insert/update/delete). Activities will include testing the creation of plans with valid information, executing field validations for required fields, string lengths and areas where numeric values occur, confirming all pop-up dialogs and navigation to the plan detail screen, and validating capacity and empty selections.

2. Class Selection and Modification

The features tested include adding and removing classes from a user's plan, filtering by class type and/or date, and modifying a user's selections. Automated UI testing verifies that a user's selection or modification of classes is reflected in the application. Edge cases, including classes that are at maximum capacity and situations in which no classes are available, are covered by automated tests.

3. Purchase Workflow

The features related to purchasing include selecting a plan, proceeding to checkout, selecting payment options, confirming payment, and processing payments that cannot be completed due to insufficient funds or invalid information provided by customers. The functional test confirms that a user is able to complete their purchase from the point they have selected the plan through to confirmation of payment. A backend verification confirms that each transaction has been correctly recorded. Activities conducted to test include validating that a purchase is possible with valid payment options; validating that the application is able to produce appropriate error messages when a user provides invalid payment options; and verifying that the correct information on the backend database is updated after the purchase is made.

4. Error Handling and Edge Cases

The features available related to validation include error messages for capacity constraints, invalid input values, past due dates, and payment errors. Negative testing confirms that all invalid inputs will generate an error message as a result, and that the application will not allow a user to continue with any actions when the constraints have not been met. Activities conducted to test include invoking validation rules and confirming that the application will generate an error message when invoking the validation rules, as well as determining whether the application will restrict actions from being performed if invalid data or conditions are used.

Techniques and Tools

The following processes should be performed/observed when testing: using Selenium WebDriver for frontend automation, using xUnit for executing tests and logging results, using database scripts for setting up the necessary test data, using code coverage analysis on back-end logic, using boundary value analysis for numeric fields, using equivalence partitioning for string inputs and payment methods, using requirement traceability to validate that all use cases are included in testing.

Completion Criteria

A successful completion of all test cases will have been accomplished at least once, all validation rules have been executed and proven, all exceptions have been handled, and updates for active plans and purchases exist and are accurate and representative of what is in the database.

Constraints

Test item availability will require the database to have classes, plans and payment options. Resource availability will require Selenium WebDriver that works with the appropriate compatible browser(s) and backend API access. All testing should meet time constraints and should not conflict with execution of other tests that are dependent on items in the database.

Estimation of Effort

1-2 Days are estimated for test setup and preparing data for testing. 3-4 Days for executing functional and negative tests. 1-2 Days for verifying coverage, logging, and addressing flaky tests.

7. Item pass/fail criteria [José Ángel]

Specify the criteria to be used to determine whether each test item has passed or failed testing.

The criteria for passing or failing any test item is very simple: If the actual result is the same as the expected result, the test is considered "passed." If the actual result is not equal to the expected result, the test is considered "failed."

8. Suspension criteria and resumption requirements [José Ángel]

Specify the criteria used to suspend all or a portion of the testing activity on the test items associated with this plan. Specify the testing activities that must be repeated, when testing is resumed.

If a critical failure occurs with the system, preventing completion of any testing work, the testing efforts will be put on hold until a permanent fix is found and no longer affects the normal functioning of the system.

When testing has resumed after an interruption, the tests that had previously been in progress or which may have been adversely impacted by the critical failure shall be repeated to guarantee that the test results accurately reflect the status of the system at that point in time.

9. Test deliverables [José Ángel]

Identify the deliverable documents. The following documents should be included:

l) Test plan;
m) Test design specifications;
n) Test case specifications;
o) Test procedure specifications;
p) Test item transmittal reports;
q) Test logs;
r) Test incident reports;
s) Test summary reports.

Test input data and test output data should be identified as deliverables. Test tools (e.g., module drivers and stubs) may also be included. (Tool used: Selenium)

Test incident reports: Any defects or issues discovered during testing are reported and tracked using GitHub Issues. Each bug includes a description, steps to reproduce, expected vs actual results and status updates.

10. Testing tasks [José Ángel]

Identify the set of tasks necessary to prepare for and perform testing. Identify all intertask dependencies and any special skills required.

The set of tasks necessary to prepare for and perform testing includes the following:

- **Test Design & Implementation**: The purpose of this task is to generate test cases and test procedures that describe how to test the product or system based on the use cases and requirements. All functional capabilities of the product must be verified with test cases and the test cases must be appropriate to allow execution within the testing procedures.
- **Test Execution:** The purpose of this task will be the execution of the test cases created in the previous task and monitoring the results of testing to compare them against the expected results and classify the result of each test as either a pass or a fail.
- **Test Environment Set-up & Maintenance:** This task includes the preparation and maintenance of the testing environment that will support the execution of testing, including anything that will need to be installed, like databases and any other dependencies that are required to complete the testing process.
- **Test Incident Reporting:** This task includes any discrepancies, errors or failures that occur during testing with enough detail for developers to be able to replicate and fix them.

The required special skills include an understanding of how to use the application/system that will be tested, experience with testing tools such as Selenium and familiarity with using GitHub for tracking and reporting any issues. Task dependencies relate to the fact that testing must be designed prior to the execution of that testing, the environment must be prepared prior to executing tests and that any incidents found during execution should be reported immediately.

11. Environmental needs

Specify both the necessary and desired properties of the test environment. This specification should contain the physical characteristics of the facilities including the hardware, the communications and system soft- ware, the mode of usage (e.g., stand-alone), and any other software or supplies needed to support the test. Also specify the level of security that must be provided for the test facilities, system software, and proprietary components such as software, data, and hardware.

Identify special test tools needed. Identify any other testing needs (e.g., publications or office space). Identify the source for all needs that are not currently available to the test group.

12. Responsibilities

Identify the groups responsible for managing, designing, preparing, executing, witnessing, checking, and resolving. In addition, identify the groups responsible for providing the test items identified in 4.2.3 and the environmental needs identified in 4.2. .

These groups may include the developers, testers, operations staff, user representatives, technical support staff, data administration staff, and quality support staff.

13. Staffing and training needs

Specify test staffing needs by skill level. Identify training options for providing necessary skills.

14. Schedule

Include test milestones identified in the software project schedule as well as all item transmittal events.

Define any additional test milestones needed. Estimate the time required to do each testing task. Specify the schedule for each testing task and test milestone. For each testing resource (i.e., facilities, tools, and staff), specify its periods of use.

15. Risks and contingencies

Identify the high-risk assumptions of the test plan. Specify contingency plans for each (e.g., delayed delivery of test items might require increased night shift scheduling to meet the delivery date).

16. Approvals

Specify the names and titles of all persons who must approve this plan. Provide space for the signatures and dates.

José Ángel Sotoca Pulpón

Daniel Félix Moya